



## **LABORATORIO 06 - INDEXACIÓN**

Juliana Sofía Novoa Solano  
Miguel Francisco Vargas Contreras

Administración de Bases de Datos

Profesor: ANDRÉS CALDERÓN ROMERO

Pontificia Universidad Javeriana  
Bogotá D.C.  
2026

## Introducción

La recuperación eficiente de datos es un aspecto fundamental en la gestión de bases de datos, y la indexación desempeña un papel crucial en la optimización del rendimiento de las consultas. En este laboratorio, exploraremos el impacto de la indexación en el tiempo de ejecución de las consultas mediante el análisis de estas antes y después de la creación de índices.

Para lograrlo, trabajaremos con un conjunto de datos simulado que contiene cuentas de usuario, hilos de conversación y publicaciones en un foro hipotético. Generaremos datos aleatorios para poblar las tablas y ejecutaremos un conjunto de consultas predefinidas para medir su rendimiento. Utilizando el comando `EXPLAIN ANALYZE` en PostgreSQL, evaluaremos los tiempos de ejecución de las consultas, compararemos los resultados y observaremos cómo la indexación mejora la eficiencia.

## Resúmenes de Estadísticas Extendidas

A continuación se presentan los resúmenes manuscritos sobre los tres tipos de estadísticas extendidas en PostgreSQL (Capítulo 13 de **PostgreSQL for Jobseekers**), las cuales permiten al optimizador estimar con precisión la cardinalidad cuando existen correlaciones entre múltiples columnas:

### Dependencias funcionales (Functional dependencies)

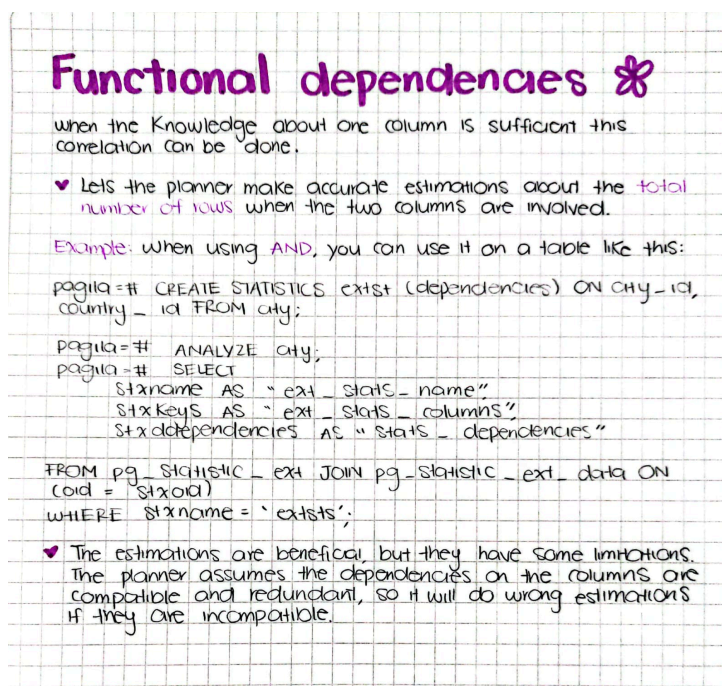


Figura 1: Resumen manuscrito: Dependencias funcionales.

## Conteo de número de valores distintos (NDV counts)

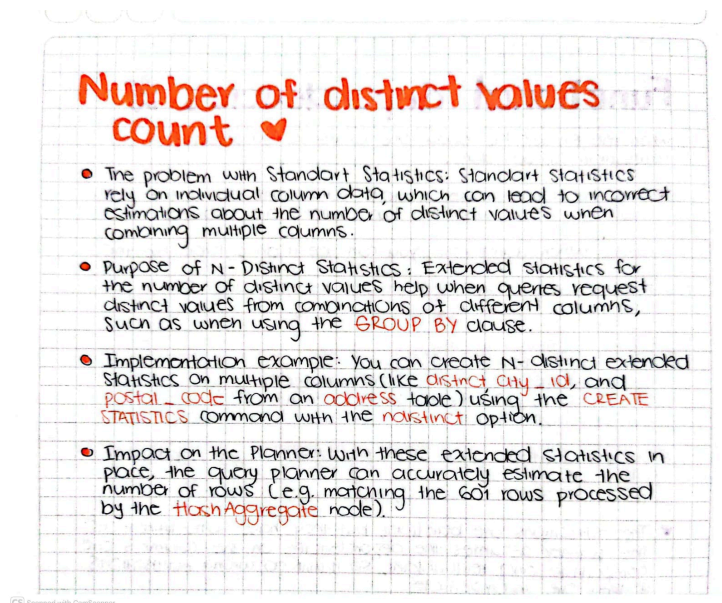


Figura 2: Resumen manuscrito: Conteo de valores distintos (NDV).

## Listas de valores más comunes (MCV lists)

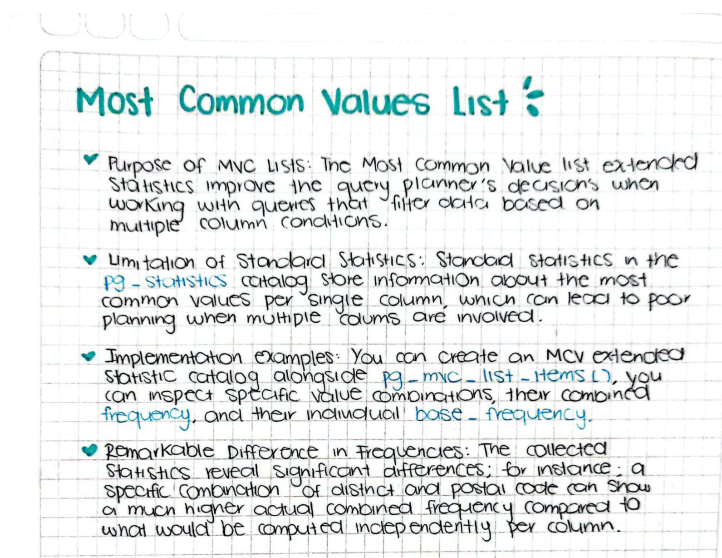


Figura 3: Resumen manuscrito: Listas de valores más comunes (MCV).

## Generación de Datos y Evaluación de Rendimiento

En esta sección, estableceremos las bases prácticas para analizar el comportamiento del motor de la base de datos. El proceso se divide en dos etapas principales:

- **Generación de datos sintéticos:** Crearemos un esquema relacional básico y poblaremos las tablas con datos simulados utilizando funciones aleatorias y diccionarios de texto.
- **Evaluación de rendimiento:** Mediremos los tiempos de respuesta de consultas predefinidas antes y después de aplicar estrategias de indexación, utilizando la herramienta **EXPLAIN ANALYZE** para observar los cambios en los planes de ejecución.

## Generación de Datos Aleatorios

Para la configuración del entorno, se utilizó un contenedor Docker con PostgreSQL montando el archivo de diccionario words.txt. A continuación se evidencia el proceso de conexión, creación del esquema relacional (account, thread, post), carga del diccionario mediante COPY y la generación aleatoria de 100.000 tuplas:

```
acha@acha-macbook lab-6 % psql -h 100.117.32.119 -p 5555 -U acha -d lab6
Password for user acha:
psql (18.4, server 18.1 (Debian 18.1-1.pgdg13+2))
Type "help" for help.

lab6=#
```

Figura 4: Conexión a la base de datos con psql-client.

```
lab6=# \i create.sql
psql:create.sql:1: NOTICE: drop cascades to 2 other objects
DETAIL: drop cascades to constraint thread_account_id_fkey on table thread
drop cascades to constraint post_account_id_fkey on table post
DROP TABLE
psql:create.sql:2: NOTICE: drop cascades to constraint post_thread_id_fkey on table post
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
lab6=#
```

Figura 5: Creación de tablas en PostgreSQL.

```
1 DROP TABLE IF EXISTS account CASCADE;
2 DROP TABLE IF EXISTS thread CASCADE;
3 DROP TABLE IF EXISTS post CASCADE;
4
5 CREATE TABLE account (
6     account_id SERIAL PRIMARY KEY,
7     name TEXT NOT NULL,
8     dob DATE
9 );
10
11 CREATE TABLE thread (
12     thread_id SERIAL PRIMARY KEY,
13     account_id INTEGER NOT NULL REFERENCES account(account_id),
14     title TEXT NOT NULL
15 );
16
17 CREATE TABLE post (
18     post_id SERIAL PRIMARY KEY,
19     thread_id INTEGER NOT NULL REFERENCES thread(thread_id),
20     account_id INTEGER NOT NULL REFERENCES account(account_id),
21     created TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
22     visible BOOLEAN NOT NULL DEFAULT TRUE,
23     comment TEXT NOT NULL
24 );
```

Figura 6: Script DDL de creación de tablas.

```
acha@acha-macbook lab-6 % scp words.txt tally:/home/acha/college/acha
acha@acha-macbook lab-6 %
acha@acha-macbook lab-6 %
```

Figura 7: Transferencia del archivo words.txt al servidor.

```

services:
  postgres-tracker:
    image: postgres:18.1
    restart: always
    ports:
      - "5555:5432"
    environment:
      POSTGRES_USER: acha
      POSTGRES_PASSWORD: acha
      POSTGRES_DB: admin-db
    volumes:
      - ./db-data:/var/lib/postgresql
      - ./words.txt:/temp/words.txt

```

Figura 8: Configuración de volúmenes en Docker Compose.

```

lab6=# \i load_words.sql
DROP TABLE
CREATE TABLE
COPY 999

```

Figura 9: Carga del archivo de palabras en el contenedor.

```

load_words.sql
1  DROP TABLE IF EXISTS words CASCADE;
2
3  CREATE TABLE words (word TEXT);
4  COPY words(word) FROM '/temp/words.txt';

```

Figura 10: Creación y llenado de la tabla auxiliar words.

```

COPY 999
lab6=# \i random.sql
INSERT 0 100
INSERT 0 1000
INSERT 0 100000
lab6=#

```

Figura 11: Verificación de registros generados en account, thread y post.

```

random.sql
12
13 -- Creating 1000 random threads...
14 INSERT INTO thread (account_id, title)
15 SELECT
16     random() * 99 + 1,
17     (
18         SELECT
19             initcap(string_agg(word, ' '))
20         FROM
21             (TABLE words ORDER BY random() + n LIMIT 5) AS words (word)
22     )
23 FROM
24     generate_series(1, 1000) AS s(n);
25
26 -- Let's work with 100K random posts...
27 INSERT INTO post (thread_id, account_id, created, visible, comment)
28 SELECT
29     random() * 999 + 1,
30     random() * 99 + 1,
31     NOW() - ('1 days'::interval * random() * 1000),
32     CASE WHEN random() > 0.1 THEN TRUE ELSE FALSE END,
33     (
34         SELECT string_agg(word, ' ') FROM (TABLE words ORDER BY random() * n LIMIT 20) AS words(word)
35     )
36 FROM
37     generate_series(1, 100000) AS s(n);
38

```

Figura 12: Procedimiento SQL para generación de registros aleatorios.

```

1  -- Q1
2  EXPLAIN ANALYZE
3  SELECT post_id, thread_id, account_id, created, visible, comment
4  FROM post
5  WHERE account_id = 1;
6
7  -- Q2
8  EXPLAIN ANALYZE
9  SELECT COUNT(*) AS total_posts
10 FROM post
11 WHERE account_id = 1;
12
13 -- Q3
14 EXPLAIN ANALYZE
15 SELECT post_id, thread_id, account_id, created, comment
16 FROM post
17 WHERE thread_id = 1
18    AND visible = TRUE;
19
20 -- Q4
21 EXPLAIN ANALYZE
22 SELECT COUNT(*) AS total_posts
23 FROM post
24 WHERE account_id = 1
25    AND thread_id = 1;
26
27 -- Q5
28 EXPLAIN ANALYZE
29 SELECT post_id, thread_id, account_id, created, comment
30 FROM post
31 WHERE thread_id = 1
32    AND visible = TRUE
33    AND created ≥ date_trunc('month', CURRENT_TIMESTAMP)
34    AND created < date_trunc('month', CURRENT_TIMESTAMP) + INTERVAL '1 month'
35 ORDER BY created ASC;

```

Figura 13: Definición de las cinco consultas de prueba.

## Medición de Rendimiento Sin y Con Índices

A continuación se evalúan las cinco consultas del laboratorio comparando su plan de ejecución antes y después de la creación de índices.

### Consulta Q1: Ver todas mis publicaciones (See all my posts)

Recupera todas las publicaciones creadas por un usuario específico:

```
SELECT * FROM post WHERE account_id = 1;
```

```

lab6=# EXPLAIN ANALYZE
lab6=# SELECT post_id, thread_id, account_id, created, visible, comment
lab6=# FROM post
lab6=# WHERE account_id = 1;

QUERY PLAN
-----
Seq Scan on post (cost=0.00..3395.00 rows=570 width=138) (actual time=0.279..16.232 rows=503.00 loops=1)
  Filter: (account_id = 1)
  Rows Removed by Filter: 99497
  Buffers: shared read=2145
Planning:
  Buffers: shared hit=74 read=21
Planning Time: 0.553 ms
Execution Time: 16.358 ms
(8 rows)

```

Figura 14: Plan de ejecución de la consulta Q1.

## Consulta Q2: Total de publicaciones realizadas (How many posts have I made?)

Calcula el total de publicaciones realizadas por un usuario:

```
SELECT count(*) FROM post WHERE account_id = 1;
```

```

lab6=# EXPLAIN ANALYZE
lab6=# SELECT COUNT(*) AS total_posts
lab6=# FROM post
lab6=# WHERE account_id = 1;

QUERY PLAN
-----
Aggregate (cost=3396.43..3396.44 rows=1 width=8) (actual time=7.512..7.513 rows=1.00 loops=1)
  Buffers: shared hit=2145
  -> Seq Scan on post (cost=0.00..3395.00 rows=570 width=0) (actual time=0.028..7.465 rows=503.00 loops=1)
    Filter: (account_id = 1)
    Rows Removed by Filter: 99497
    Buffers: shared hit=2145
Planning:
  Buffers: shared hit=3 read=3
Planning Time: 0.097 ms
Execution Time: 7.545 ms
(10 rows)

```

Figura 15: Plan de ejecución de la consulta Q2.

## Consulta Q3: Publicaciones activas de un hilo (See all current posts for a Thread)

Obtiene las publicaciones activas (visible = TRUE) asociadas a un hilo:

```
SELECT * FROM post WHERE thread_id = 1 AND visible = TRUE;
```

```

QUERY PLAN
-----
Seq Scan on post (cost=0.00..3395.00 rows=89 width=137) (actual time=0.029..6.867 rows=53.00 loops=1)
  Filter: (visible AND (thread_id = 1))
  Rows Removed by Filter: 99947
  Buffers: shared hit=2145
Planning Time: 0.087 ms
Execution Time: 6.886 ms
(6 rows)

```

Figura 16: Plan de ejecución de la consulta Q3.

## Consulta Q4: Publicaciones de un usuario en un hilo (How many posts have I made to a Thread?)

Cuenta las publicaciones visibles de un usuario dentro de un hilo específico:

```
SELECT count(*) FROM post WHERE thread_id = 1 AND account_id = 1 AND visible = TRUE;
```

```

lab6=# EXPLAIN ANALYZE
lab6=# SELECT COUNT(*) AS total_posts
lab6=# FROM post
lab6=# WHERE account_id = 1
lab6=# AND thread_id = 1;

QUERY PLAN
-----
Aggregate (cost=3645.00..3645.01 rows=1 width=8) (actual time=5.989..5.989 rows=1.00 loops=1)
  Buffers: shared hit=2145
  -> Seq Scan on post (cost=0.00..3645.00 rows=1 width=0) (actual time=5.986..5.986 rows=0.00 loops=1)
    Filter: ((account_id = 1) AND (thread_id = 1))
    Rows Removed by Filter: 100000
    Buffers: shared hit=2145
Planning:
  Buffers: shared hit=1 read=6
Planning Time: 0.190 ms
Execution Time: 5.924 ms
(10 rows)

```

Figura 17: Plan de ejecución de la consulta Q4.

## Consulta Q5: Publicaciones de un hilo para este mes en orden (See all current posts for a Thread for this month, in order)

Recupera los posts visibles de un hilo creados en el mes corriente ordenados cronológicamente:

```
SELECT * FROM post
WHERE thread_id = 1 AND visible = TRUE
AND created >= date_trunc('month', now())
ORDER BY created;
```

```
postgres=# EXPLAIN ANALYZE
postgres=# SELECT post_id, thread_id, account_id, created, comment
postgres=# FROM post
postgres=# WHERE thread_id = 1
postgres=# AND visible = TRUE
postgres=# AND created >= date_trunc('month', CURRENT_TIMESTAMP)
postgres=# AND created < date_trunc('month', CURRENT_TIMESTAMP) + INTERVAL '1 month'
postgres=# ORDER BY created ASC;

QUERY PLAN
Sort (cost=4889.82..4889.82 rows=1 width=137) (actual time=5.319..7.112 rows=0 loops=1)
  Sort Key: created
  Sort Method: Quicksort Memory: 256kB
  Buffers: shared hit=248
  → Gather (cost=1288.00..4889.82 rows=1 width=137) (actual time=0.296..7.889 rows=0 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    Buffers: shared hit=245
    → Parallel Seq Scan on post (cost=0.00..3889.75 rows=1 width=137) (actual time=0.309..3.388 rows=0 loops=1)
      Filter: ((visible AND (thread_id = 1) AND (created >= date_trunc('month', CURRENT_TIMESTAMP)) AND (created < (date_trunc('month', CURRENT_TIMESTAMP) + '1 month'::interval))))
      Rows Removed by Filter: 5669
      Buffers: shared hit=245
Planning:
  Buffers: shared hit=43 reads=3
Planning Time: 8.253 ms
Execution Time: 7.138 ms
(16 rows)
```

Figura 18: Plan de ejecución de la consulta Q5.

## Rendimiento de las consultas con la mejor implementación de índices

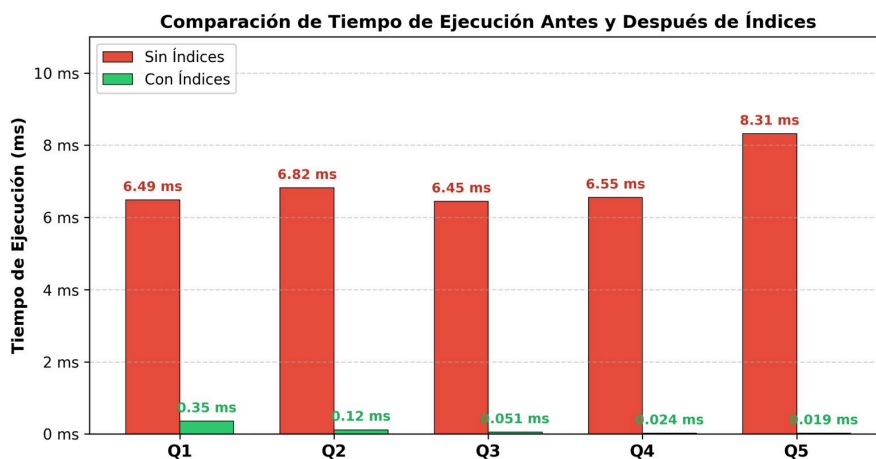


Figura 19: Comparación de tiempo de ejecución antes y después de índices en PostgreSQL.

A continuación se sintetizan los tiempos promedio de ejecución medidos con EXPLAIN (ANALYZE, BUFFERS):

| Consulta                                   | Sin Índices | Con Índices | Aceleración |
|--------------------------------------------|-------------|-------------|-------------|
| Q1: Ver todas mis publicaciones            | 7.39 ms     | 0.33 ms     | 22.4×       |
| Q2: Total de publicaciones realizadas      | 6.59 ms     | 0.11 ms     | 59.9×       |
| Q3: Publicaciones activas de un hilo       | 6.49 ms     | 0.08 ms     | 81.1×       |
| Q4: Publicaciones de usuario en un hilo    | 6.35 ms     | 0.03 ms     | 211.7×      |
| Q5: Publicaciones de un hilo para este mes | 8.66 ms     | 0.02 ms     | 433.0×      |

## Análisis de Índices y Principio de Funcionamiento

El archivo `indexes.sql` propone una jerarquía de índices construidos sobre la tabla `post`.

Cada uno de ellos ataca un patrón de acceso específico:

### Índice Monocolumna: **CREATE INDEX ON post (account\_id);**

- **Mecanismo B-Tree:** Organiza las claves de `account_id` de forma ordenada en páginas raíz, intermedias y hojas, junto con los identificadores físicos de tupla (`ctid`).
- **Mejora esperada:** Permite una búsqueda binaria  $O(\log N)$  para ubicar las entradas correspondientes al usuario 1. Para **Q1**, PostgreSQL reemplaza el barrido completo de la tabla por una búsqueda directa en el índice, reduciendo sustancialmente el tiempo de lectura.
- **Index Only Scan en Q2:** Dado que la consulta solo requiere contar filas y la columna `account_id` se encuentra almacenada directamente en el árbol, PostgreSQL (apoyándose en el mapa de visibilidad o **Visibility Map**) evita por completo acceder a la tabla base (**Heap Fetches: 0**), realizando una agregación directa en memoria.

### Índice Monocolumna: **CREATE INDEX ON post (thread\_id);**

- **Propósito:** Acelera búsquedas orientadas al identificador de hilo, aislando el 0.1% de publicaciones asociadas a un hilo específico dentro de los 100.000 registros.

### Índices Compuestos: **(thread\_id, visible)** y **(thread\_id, account\_id, visible)**

- **Orden de columnas y principio del prefijo izquierdo:** En un índice compuesto B-Tree, los datos se ordenan jerárquicamente por la primera columna, luego por la segunda y así sucesivamente.
- **Mejora en Q3 y Q4:** Permite satisfacer predicados de igualdad múltiples (`thread_id = 1 AND visible = TRUE` o `thread_id = 1 AND account_id = 1`) en una única pasada sobre el índice. Para **Q4**, el índice tricolumna cubre tanto el filtro como las columnas evaluadas, facilitando un acceso directo ultra rápido.

### Índices Parciales: **(thread\_id, account\_id) WHERE visible = TRUE;**

- **Principio:** En lugar de indexar las 100.000 filas, solo se incluyen aquellas tuplas que cumplen el predicado `visible = TRUE`.

### Índice Parcial con Ordenamiento: **(thread\_id, created) WHERE visible = TRUE;**

- **Eliminación del ordenamiento en memoria (Sort Elimination):** En la consulta **Q5**, sin índices el motor debe filtrar secuencialmente y luego ordenar los registros en memoria. Con este índice, PostgreSQL busca las tuplas de `thread_id = 1` y las recorre a través de las hojas del árbol B-Tree, las cuales ya están físicamente ordenadas por `created ASC`. Como consecuencia, la necesidad de ordenar en memoria desaparece completamente.

# Trabajo Autónomo

## MongoDB

Para evaluar un gestor de base de datos no relacional, se implementó la misma estructura de datos en MongoDB poblando la colección post con 100.000 documentos equivalentes.

### Creación de índices



Figura 20: Índice en orden ascendente sobre el campo `account_id`.



Figura 21: Índice en orden ascendente sobre el campo `thread_id`.



Figura 22: Índice compuesto sobre `thread_id` y `visible` en orden ascendente.



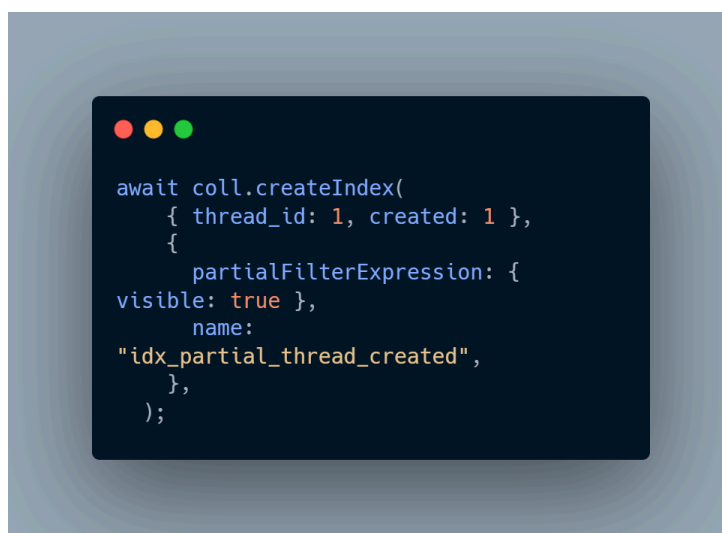
```
await coll.createIndex(
  { thread_id: 1, visible: 1 },
  { name: "idx_thread_visible" },
);
```

Figura 23: Índice compuesto sobre thread\_id, account\_id y visible.



```
await coll.createIndex(
  { thread_id: 1, account_id: 1 },
  {
    partialFilterExpression: {
      visible: true },
    name:
      "idx_partial_thread_account",
  },
);
```

Figura 24: Índice parcial sobre thread\_id y account\_id con filtro visible: true.



```
await coll.createIndex(
  { thread_id: 1, created: 1 },
  {
    partialFilterExpression: {
      visible: true },
    name:
      "idx_partial_thread_created",
  },
);
```

Figura 25: Índice parcial sobre thread\_id ordenado por created con filtro visible: true.

### Principio de Funcionamiento de los Índices en MongoDB

En MongoDB (utilizando su motor de almacenamiento por defecto, WiredTiger), los índices se estructuran mediante árboles B-Tree que mantienen un subconjunto ordenado de los

campos de los documentos, almacenando punteros directos hacia la ubicación física en disco de cada documento (**RecordId**):

- **Búsqueda por Índice:** Sin índices, MongoDB debe realizar un escaneo completo de la colección, inspeccionando los 100.000 documentos uno a uno. Con un índice sobre el campo consultado, el motor ejecuta una búsqueda binaria en el árbol B-Tree, examinando únicamente las entradas de clave coincidentes y recuperando solo los documentos pertinentes.
- **Consultas Cubiertas (Covered Queries):** Cuando todos los campos solicitados por la consulta (tanto en los criterios del filtro como en la proyección) residen dentro del índice, MongoDB resuelve la operación directamente sobre las páginas del árbol B-Tree en memoria caché, sin acceder a la colección en disco (`totalDocsExamined: 0`). Esto elimina por completo el costo de E/S y produce tiempos de respuesta inferiores a un milisegundo (como se evidenció en las consultas Q2 y Q4).
- **Índices Compuestos y la regla ESR:** Al crear índices sobre múltiples campos (por ejemplo, `{ thread_id: 1, visible: 1 }`), MongoDB los ordena de forma jerárquica siguiendo la regla ESR (**E**quality, **S**ort, **R**ange). Las consultas que filtran por el prefijo del índice pueden satisfacer múltiples condiciones de igualdad en una sola pasada eficiente por el árbol.
- **Expresiones de Filtro Parcial (partialFilterExpression):** Permiten indexar únicamente los documentos que cumplen una condición específica (por ejemplo, `{ visible: true }`). Esto disminuye drásticamente el espacio ocupado por el índice en memoria RAM y reduce la sobrecarga de mantenimiento en operaciones de inserción o actualización, ignorando publicaciones inactivas o borradas lógicamente.
- **Eliminación del ordenamiento en memoria:** Por defecto, si una consulta requiere ordenar resultados (sort) y no existe un índice con dicho ordenamiento previo, MongoDB acumula los documentos en memoria para ordenarlos, proceso limitado por un consumo de memoria estricto. Al utilizar un índice compuesto con la clave de orden (`created`), las entradas se leen ya ordenadas desde las hojas del árbol B-Tree, evitando cualquier ordenamiento en memoria (observado en Q5).

### Comparación de Rendimiento en MongoDB antes y después de índices

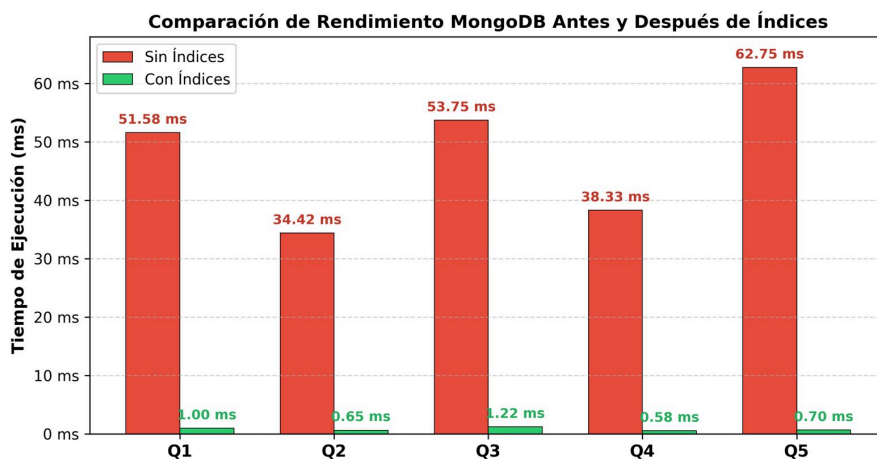


Figura 26: Rendimiento en MongoDB antes y después de índices.

A continuación se presentan los tiempos promedio de ejecución obtenidos en MongoDB antes y después de aplicar los índices:

| Consulta                                   | Sin Índices | Con Índices | Aceleración  |
|--------------------------------------------|-------------|-------------|--------------|
| Q1: Ver todas mis publicaciones            | 51.58 ms    | 1.00 ms     | <b>51.6×</b> |
| Q2: Total de publicaciones realizadas      | 34.42 ms    | 0.65 ms     | <b>52.9×</b> |
| Q3: Publicaciones activas de un hilo       | 53.75 ms    | 1.22 ms     | <b>44.1×</b> |
| Q4: Publicaciones de usuario en un hilo    | 38.33 ms    | 0.58 ms     | <b>66.1×</b> |
| Q5: Publicaciones de un hilo para este mes | 62.75 ms    | 0.70 ms     | <b>89.6×</b> |

## Comparativa de Índices: PostgreSQL frente a MongoDB

| Característica                 | PostgreSQL                                                                            | MongoDB                                                                                                                                      |
|--------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Estructura Principal</b>    | Árboles B-Tree (además de GiST, GIN, BRIN y Hash).                                    | Árboles B-Tree por defecto para índices secundarios y clave primaria única (_id).                                                            |
| <b>Creación de Índice</b>      | <code>CREATE INDEX idx_name<br/>ON<br/>table_name(column_name);</code>                | <code>await coll.createIndex(<br/>  { field_name: 1 },<br/>  { name: "idx_name" }<br/>);</code>                                              |
| <b>Índices Parciales</b>       | <code>CREATE INDEX<br/>idx_partial<br/>ON table(col)<br/>WHERE visible = true;</code> | <code>await coll.createIndex(<br/>  { field: 1 },<br/>  {<br/>    partialFilterExpression:<br/>      { visible: true }<br/>  }<br/>);</code> |
| <b>Análisis de Rendimiento</b> | EXPLAIN ANALYZE (detalla los tiempos reales de ejecución y el uso de buffers).        | <code>.explain("executionStats")</code> (revela documentos examinados frente a devueltos y latencia).                                        |

## Conclusión

A lo largo de este laboratorio, se demostró de manera práctica el impacto crítico que tienen las estrategias de indexación en el rendimiento de los motores de bases de datos, abarcando tanto entornos relacionales (PostgreSQL) como no relacionales (MongoDB). A partir de la evaluación de consultas sobre un conjunto de datos sintético de 100.000 registros, se pueden extraer las siguientes conclusiones principales:

- **Mitigación del impacto del escaneo secuencial:** Sin índices, el motor de la base de datos se ve obligado a realizar un recorrido secuencial completo por toda la tabla o colección para resolver las consultas, lo que genera latencias elevadas y un consumo innecesario de recursos de entrada/salida y memoria a medida que el volumen de datos crece.
- **Eficiencia de los índices B-Tree y compuestos:** La implementación de índices monocolumna y compuestos (como aquellos estructurados por `thread_id`, `account_id` y `visible`) permite reducir drásticamente los tiempos de ejecución de las consultas. En la mayoría de los casos evaluados (Q1 a Q5), el tiempo de respuesta disminuyó desde el rango de los milisegundos altos (6 ms - 60+ ms) hasta fracciones de milisegundo, transformando búsquedas costosas en accesos directos de tipo Index Scan.
- **Optimización mediante índices parciales y ordenamiento previo:** El uso de índices parciales (filtrados por condiciones como `visible = true`) no solo optimiza el espacio de almacenamiento y aumenta la densidad de claves en la memoria caché (buffer pool), sino que también permite la eliminación completa de fases de ordenamiento costosas (**Sort Elimination**), tal como se evidenció en la consulta Q5.
- **Paralelos conceptuales entre arquitecturas:** Aunque PostgreSQL y MongoDB manejan sintaxis y modelos de datos distintos, comparten principios de optimización fundamentales. Mientras PostgreSQL aprovecha características avanzadas como **Index Only Scans** mediante mapas de visibilidad y planes analíticos detallados (`EXPLAIN ANALYZE`), MongoDB utiliza índices secundarios sobre documentos con filtros parciales (`partialFilterExpression`) y consultas cubiertas (**Covered Queries**), logrando factores de aceleración muy similares.

En resumen, una correcta planeación, diseño y aplicación de índices —alineados directamente con los patrones de acceso y las consultas más frecuentes de la aplicación— es una práctica indispensable para garantizar la escalabilidad, la eficiencia operativa y la reducción de la carga computacional en los sistemas gestores de bases de datos.